DSP Lab Mini Project – Adaptive Channel Equalizer (LMS)

MATLAB code, command window output and figures · Fall 2026 / 2027 · Dr. Samir Kamal

1. MATLAB code — adaptive_equalizer.m

```
%% DSP Mini Project : Adaptive Channel Equalizer using the LMS algorithm
% A random BPSK sequence (+1/-1) is sent through a channel that causes
% inter-symbol interference (ISI) plus white Gaussian noise.
% An 11-tap FIR filter adapts its weights with the LMS algorithm so that
% its output becomes equal to the transmitted (delayed) symbol.
clc; clear; close all;

%% 1) Parameters
N     = 1000;          % number of training symbols (iterations)
M     = 11;            % number of equalizer taps
mu    = 0.075;         % LMS step size
W     = 3.1;           % channel parameter (bigger W = more distortion)
SNRdB = 30;            % signal to noise ratio in dB
runs  = 200;           % independent runs to average the learning curve
delay = 1 + (M-1)/2;   % total delay = channel delay (1) + equalizer centre (5) = 6

%% 2) Channel impulse response (raised cosine)
h = channel(W);                          % h = [h1 h2 h3]
disp('Channel impulse response h(n) = '); disp(h)

%% 3) Learning curve (MSE averaged over many runs)
MSE = zeros(1,N);
for r = 1:runs
    [e, w] = lms_equalizer(h, N, M, mu, delay, SNRdB);
    MSE = MSE + e.^2;
end
MSE = MSE / runs;

%% 4) One run to look at the signals before and after the equalizer
a     = sign(randn(1,N));                          % transmitted symbols
x     = filter(h, 1, a);                           % channel output (with ISI)
u     = x + sqrt(10^(-SNRdB/10)) * randn(1,N);     % received signal (with noise)
yeq   = filter(w, 1, u);                           % equalizer output (final taps)

%% 5) Figures
% Figure 1 : channel
figure(1)
subplot(2,1,1); stem(0:2, h, 'filled'); grid on
title('Channel impulse response h(n)'); xlabel('n'); ylabel('h(n)')
[Hc, f] = freqz(h, 1, 512);
subplot(2,1,2); plot(f/pi, 20*log10(abs(Hc))); grid on
title('Channel frequency response'); xlabel('\omega / \pi'); ylabel('|H| (dB)')

% Figure 2 : signals before and after equalization
figure(2)
k = N-100:N;
subplot(3,1,1); stem(k, a(k), '.'); title('Transmitted symbols a(n)'); ylim([-2 2])
subplot(3,1,2); stem(k, u(k), '.'); title('Received signal u(n) (ISI + noise)'); ylim([-2 2])
subplot(3,1,3); stem(k, yeq(k), '.'); title('Equalizer output y(n)'); ylim([-2 2]); xlabel('n')

% Figure 3 : learning curve
figure(3)
semilogy(M:N, MSE(M:N)); grid on
title(['Learning curve of the LMS equalizer (\mu = ' num2str(mu) ')'])
xlabel('iteration n'); ylabel('mean square error')

% Figure 4 : effect of the step size mu
figure(4)
mus = [0.0075 0.025 0.075];
for i = 1:length(mus)
    MSEi = zeros(1,N);
    for r = 1:runs
        e = lms_equalizer(h, N, M, mus(i), delay, SNRdB);
        MSEi = MSEi + e.^2;
    end
    semilogy(M:N, MSEi(M:N)/runs); hold on
end
grid on; legend('\mu = 0.0075', '\mu = 0.025', '\mu = 0.075')
title('Effect of the step size \mu'); xlabel('iteration n'); ylabel('mean square error')

% Figure 5 : effect of the channel distortion W (eigenvalue spread)
figure(5)
Ws = [2.9 3.1 3.3 3.5];
for i = 1:length(Ws)
    hi = channel(Ws(i));
    MSEi = zeros(1,N);
```

```
    for r = 1:runs
        e = lms_equalizer(hi, N, M, 0.025, delay, SNRdB);
        MSEi = MSEi + e.^2;
    end
    semilogy(M:N, MSEi(M:N)/runs); hold on
    % eigenvalue spread of the input correlation matrix
    r0 = sum(hi.^2) + 10^(-SNRdB/10);
    r1 = hi(1)*hi(2) + hi(2)*hi(3);
    r2 = hi(1)*hi(3);
    R  = toeplitz([r0 r1 r2 zeros(1,M-3)]);
    fprintf('W = %.1f   eigenvalue spread = %.2f\n', Ws(i), max(eig(R))/min(eig(R)));
end
grid on; legend('W = 2.9', 'W = 3.1', 'W = 3.3', 'W = 3.5')
title('Effect of the channel distortion W (\mu = 0.025)'); xlabel('iteration n'); ylabel('mean square error')

% Figure 6 : equalizer taps and channel + equalizer response
figure(6)
subplot(2,1,1); stem(0:M-1, w, 'filled'); grid on
title('Final equalizer taps w(k)'); xlabel('k')
subplot(2,1,2); stem(0:M+1, conv(h, w), 'filled'); grid on
title('Channel * equalizer (should be a delayed impulse)'); xlabel('n')

% Figure 7 : frequency responses
figure(7)
[He, f] = freqz(w, 1, 512);
plot(f/pi, 20*log10(abs(Hc)), f/pi, 20*log10(abs(He)), f/pi, 20*log10(abs(Hc.*He))); grid on
legend('channel H', 'equalizer W', 'H \times W')
title('Frequency responses'); xlabel('\omega / \pi'); ylabel('magnitude (dB)')

% Figure 8 : bit error rate with and without the equalizer
figure(8)
SNRs = 0:2:20;  Nb = 20000;  hb = channel(3.5);
BER0 = zeros(size(SNRs));  BER1 = zeros(size(SNRs));
for i = 1:length(SNRs)
    [~, wb] = lms_equalizer(hb, 2000, M, 0.01, delay, SNRs(i));   % train
    a  = sign(randn(1,Nb));
    u  = filter(hb, 1, a) + sqrt(10^(-SNRs(i)/10)) * randn(1,Nb);
    d0 = sign(u(M+1:end));                      % no equalizer (channel delay = 1)
    BER0(i) = mean(d0 ~= a(M:end-1));
    y  = filter(wb, 1, u);
    d1 = sign(y(M+1:end));                      % with equalizer (delay = 6)
    BER1(i) = mean(d1 ~= a(M+1-delay:end-delay));
end
semilogy(SNRs, BER0, 'o-', SNRs, BER1, 's-'); grid on
legend('without equalizer', 'with LMS equalizer')
title('Bit error rate (W = 3.5)'); xlabel('SNR (dB)'); ylabel('BER')

%% ---------------- local functions ----------------
function h = channel(W)
% raised cosine channel  h(n) = 0.5*(1 + cos(2*pi/W*(n-2))) , n = 1,2,3
n = 1:3;
h = 0.5 * (1 + cos(2*pi/W*(n-2)));
end

function [e, w] = lms_equalizer(h, N, M, mu, delay, SNRdB)
% trains an M-tap equalizer with LMS and returns the error e(n) and final taps w
a = sign(randn(1,N));                           % training symbols
u = filter(h, 1, a) + sqrt(10^(-SNRdB/10)) * randn(1,N);
w = zeros(M,1);                                 % start from zero weights
e = zeros(1,N);
for n = M:N
    un   = u(n:-1:n-M+1).';      % tap-input vector [u(n) ... u(n-M+1)]
    y    = w.' * un;             % equalizer output
    d    = a(n-delay);           % desired response = delayed symbol
    e(n) = d - y;                % error
    w    = w + mu * e(n) * un;   % LMS weight update
end
end
```

2. Command window output

```
Channel impulse response h(n) =
    0.2798    1.0000    0.2798

W = 2.9 eigenvalue spread = 6.08
W = 3.1 eigenvalue spread = 11.12
W = 3.3 eigenvalue spread = 21.71
W = 3.5 eigenvalue spread = 46.82
```

3. Figures (results)

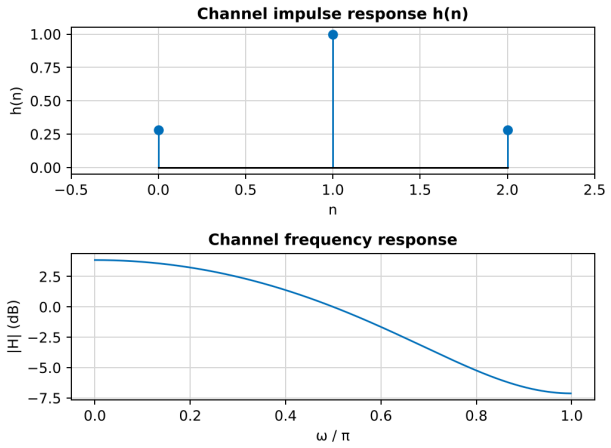


Figure 1 – Channel impulse response and frequency response (W = 3.1)

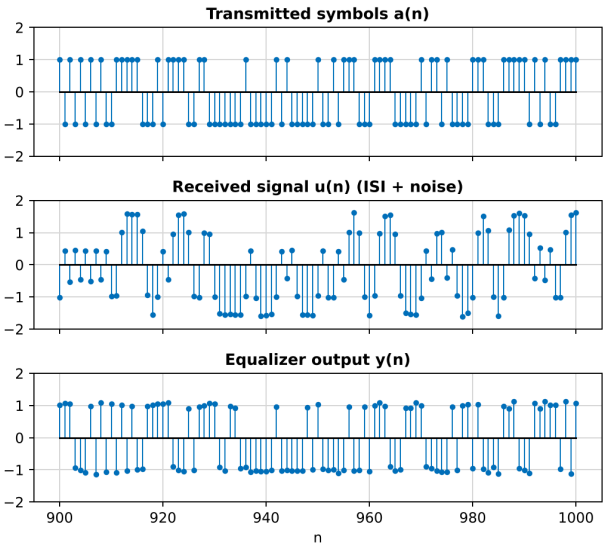


Figure 2 – Transmitted, received and equalized signals

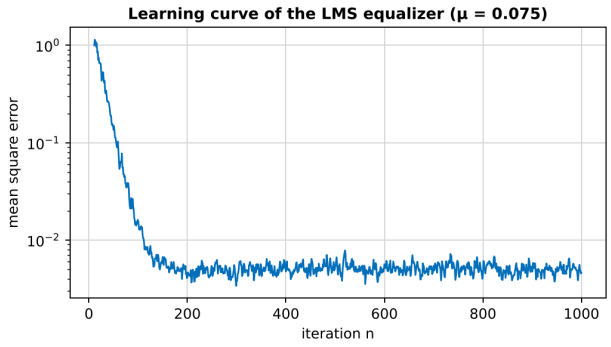


Figure 3 – Learning curve (μ = 0.075, 200 runs)

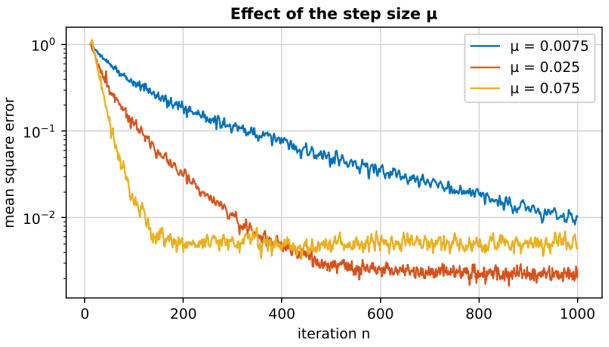


Figure 4 – Effect of the step size μ

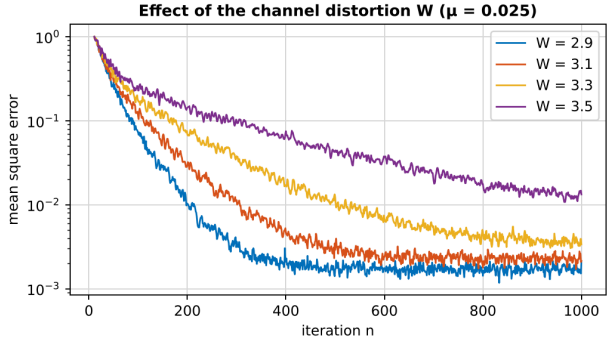


Figure 5 – Effect of the channel distortion W (μ = 0.025)

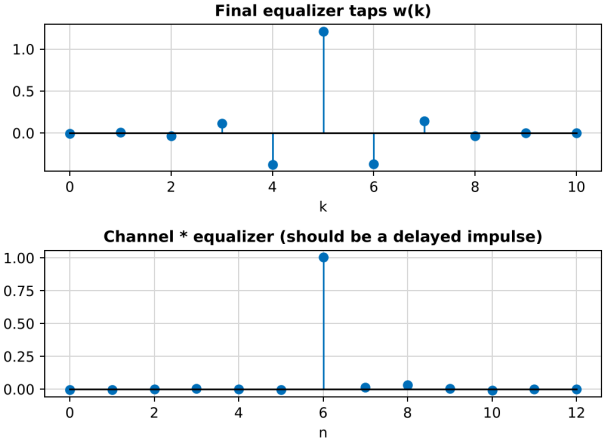


Figure 6 – Final equalizer taps and h * w

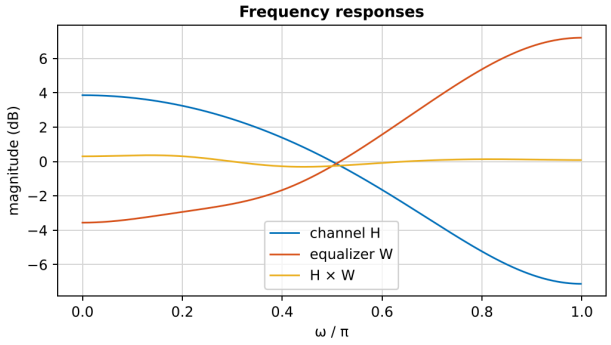


Figure 7 – Frequency responses of channel, equalizer and product

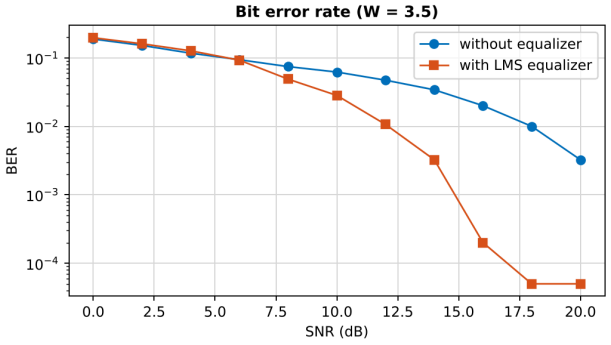


Figure 8 – Bit error rate with and without the equalizer (W = 3.5)